Step 2 — A Real Terminal Chat With Memory and Live Streaming — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — Where the memory really lives
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — Where the memory really lives

The model remembers nothing between calls (Doc03). The memory is **your list**, `history`: a system message first, then every user turn and every reply, in order. Each call sends the whole list again.

So a chat is just a loop:

- read a line,
- stop on `/quit`,
- otherwise call `stream_message(client, history, line)` — which adds both the user turn and the reply to `history` for you.

For streaming, Doc04's `stream_message()` goes into `models/llm.py`, next to `send_message()`. Add `flush=True` to its `print(piece, end="")` — without it, the terminal holds the text back until the line ends, so nothing looks streamed.

Three new small files, each with one job:

- `prompts/concierge.txt` — the system prompt, as plain text.
- `utils/prompts.py` — `load_prompt(name)` builds the path `"src/prompts/" + name + ".txt"`, opens it with `with open(path) as prompt_file:` (as in Doc01), and returns its text. The path starts at the repo root, which is where every command runs from.
- `utils/history.py` — two functions:

```
def estimate_tokens(history: list[dict]) -> int:
    # add up len(message["content"]) with a for loop, then // 4
    ...

def trim_history(history: list[dict], max_tokens: int) -> list[dict]:
    # while too big AND len(history) > 2: del history[1]
    ...
```

`del history[1]` drops the *oldest* real turn — `history[0]` is the system prompt and must stay. `len(history) > 2` stops the loop before it deletes the newest message.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
main():
    (Step 1's startup: config, logger, client, key check)
    history = [system message]
    loop:
        message = input("You: ").strip()
        if message == "/quit": stop
        if message == "": skip to the next loop
        print("Assistant: ", end="", flush=True)
        stream_message(client, history, message)
```

Test it: tell it your name, ask something else, then ask "what's my name?".

Then add two things to that plan:

```
MAX_HISTORY_TOKENS = 3000        # far below the real 128,000 limit

history = [{"role": "system", "content": load_prompt("concierge")}]
...
    trim_history(history, MAX_HISTORY_TOKENS)     # before every call
    stream_message(client, history, message)
```

Then write `tests/unit/test_history.py` with Step 1's test pattern — small `test_...()` functions that print `OK` or `WRONG`, all called under `if __name__ == "__main__":`. Three cases are enough:

- a known history → the right estimate (40 characters → 10);
- a short history → nothing removed;
- 5 long turns with a small budget → the system prompt and the newest turn survive.

Run it with `python -m tests.unit.test_history`. It needs no API key.

Full solution: Show me the solution